Documentação de Arquitetura — Plataforma de Telemetria e Gestão de Entregas

Disciplina: Arquitetura de Soluções / Computação em Nuvem — PUC

Grupo: Fernando Padilha, Hector Saldivar, Theo Otto, Vinicius Veiga

Projeto (PJBL): Plataforma de telemetria veicular (OBD2 + GPS) integrada a app mobile e a um dashboard de gestão de entregas para transportadoras.

Entrega: Etapa 2 — evolução da arquitetura para Serverless, Microfrontends, Backend for Frontend, Microsserviços com Database per Service, API Gateway, Arquitetura Orientada a Eventos, Clean Architecture com Vertical Slice e Testes Unitários de Arquitetura. Documentação Arc42 (11 seções), C4 Model (níveis 1 a 4), UML, DER e Software Architecture Canvas.

Repositório: https://github.com/ViniciusVeiga2002/pjbl_arq_cloud

Branch desta etapa: `refactor/backend-vertical-slice-clean-architecture`

Ambiente publicado: https://kind-water-063145a0f.7.azurestaticapps.net

Sobre o arc42

arc42, o template para documentação de software e arquitetura de sistemas.

Versão do template 8.2-PT (baseada na versão AsciiDoc), setembro de 2024.

Criado, mantido e © pelo Dr. Peter Hruschka, Dr. Gernot Starke e colaboradores. Veja https://arc42.org.

Este documento foi estruturado a partir do template oficial arc42 8.2-PT. A seção 12 (Glossário) foi omitida propositalmente — a entrega considera as seções 1 a 11.

Sumário

Sobre o arc42 ..... ***2***

1. Introdução e Objetivos ***4***

1.1. Visão Geral dos Requisitos **4**

1.2. Objetivos de Qualidade **6**

1.3. Partes Interessadas **7**

1.4. Software Architecture Canvas **8**

2. Restrições da Arquitetura ***10***

2.1. Restrições Técnicas **10**

2.2. Restrições Organizacionais **10**

2.3. Convenções **11**

3. Contexto e Escopo ***13***

3.1. Contexto de Negócio **13**

3.2. Contexto Técnico **14**

4. Estratégia de Solução ***15***

5. Visão de Blocos de Construção ***17***

5.1. Nível 1 — Sistema (caixa-branca) = C4 Nível 2 (Contêineres) **17**

5.2. Nível 2 — Contêineres de backend (caixa-branca) = C4 Nível 3 (Componentes) **20**

5.3. Nível 3 — Detalhes **24**

5.3.1. Modelo de classes do domínio (UML — Classes) 24

5.3.2. Componentes do frontend em microfrontends 25

5.4. Relação com o modelo de dados **26**

6. Visão de Tempo de Execução ***27***

6.1. Cenário 1 — Coordenador cadastra um pedido **27**

6.2. Cenário 2 — Concluir entrega (eventos em paralelo) **28**

6.3. Cenário 3 — Telemetria OBD2/GPS até o painel da frota **29**

6.4. Primeira invocação (cold start) e modo offline 29
7. Visão de Implantação 30
8. Conceitos Transversais 33
8.1. Modelo de dados 33
8.2. Configuração e segredos 34
8.3. Persistência em ambiente serverless 35
8.4. Tratamento de erros e contrato de resposta 35
8.5. CORS 35
8.6. Autenticação e autorização 35
8.7. Resiliência no cliente 35
8.8. Identificadores 35
8.9. Build e entrega contínua 36
8.10. Clean Architecture e Vertical Slice 36
8.11. Testes Unitários de Arquitetura 37
8.12. Eventos 38
8.13. Observabilidade 38
9. Decisões de Arquitetura 40
10. Requisitos de Qualidade 43
10.1. Árvore de Qualidade 43
10.2. Cenários de Qualidade 43
11. Riscos e Dívida Técnica 46

1. Introdução e Objetivos

A plataforma tem como objetivo de negócio dar a uma transportadora visibilidade em tempo real da frota e das entregas e reduzir falhas e atrasos na operação logística. O produto completo combina: (a) captura de telemetria do veículo via OBD2 e GPS, com funcionamento offline e posterior envio ao servidor; (b) um app mobile para o motorista consultar as entregas do dia e registrar a prova de entrega (POD); e (c) um dashboard web para o coordenador acompanhar mapa, pedidos e alertas, e para o diretor consultar indicadores.

Esta documentação descreve a arquitetura-alvo do produto (seções 1–4) e detalha em profundidade o que já está implementado (seções 5–8): um dashboard web composto por microfrontends, que acessa o sistema exclusivamente por um API Gateway; um Backend for Frontend que compõe dois microsserviços serverless — Pedidos e Frota —, cada um com o seu próprio banco de dados; e a colaboração entre os microsserviços por eventos em filas do Azure.

1.1. Visão Geral dos Requisitos

Req.	Descrição	Persona	Situação nesta entrega
RF01	Capturar dados do veículo via OBD2 (RPM, velocidade, temperatura, DTC)	Motorista	Parcial (leituras simuladas por scripts/simular-telemetria.js, publicadas em fila)
RF02	Capturar posição GPS do veículo	Motorista	Parcial (posição GPS recebida pela fila telemetria-veiculos; origem simulada)
RF03	Armazenar dados no dispositivo quando sem internet (offline)	Motorista	Planejado
RF04	Enviar os dados ao servidor quando a conexão retornar	Motorista	Planejado
RF05	Cadastrar e manter pedidos (criar, pesquisar, alterar, excluir)	Coordenador	Implementado

Req.	Descrição	Persona	Situação nesta entrega
RF06	Exibir mapa da frota em tempo real	Coordenador	Implementado (posição e velocidade reais recebidas por telemetria; mapa gráfico ainda em forma de tabela)
RF07	Vincular pedido à entrega/viagem e listar pedidos por entrega	Coordenador	Implementado (pedidos por entrega + situação consolidada da entrega no serviço de frota)
RF08	Alertar excesso de velocidade	Coordenador	Implementado (regra de excesso de velocidade > 80 km/h no domínio Veiculo)
RF09	Alertar falha mecânica (código DTC)	Coordenador	Planejado
RF10	Alertar desvio de rota	Coordenador	Planejado
RF11	Registrar prova de entrega (POD) pelo motorista	Motorista	Planejado
RF12	Manter histórico de trajeto por veículo/entrega	Coordenador	Planejado
RF13	Gerar relatórios e KPIs gerenciais (custo, falhas, prazos)	Diretor	Planejado
RF14	Controlar acesso por perfil (motorista, coordenador, diretor)	Todos	Planejado

1.2. Objetivos de Qualidade

Os cinco objetivos de qualidade que mais influenciam a arquitetura (vocabulário ISO/IEC 25010). Cenários concretos estão na seção 10.

#	Objetivo de qualidade	Motivação (dirigida pelas personas)
Q1	Disponibilidade / tolerância a falhas	A operação de campo tem internet instável; o dashboard e o app não podem "quebrar" quando a API ou o banco estiverem indisponíveis.
Q2	Eficiência de custo	Projeto acadêmico: toda a solução deve caber em planos gratuitos (Azure Static Web Apps Free, MongoDB Atlas M0), sem custo recorrente.
Q3	Manutenibilidade / simplicidade	Time de 4 pessoas: cada mudança afeta uma fatia de um serviço; as regras de arquitetura são verificadas automaticamente por testes a cada push.
Q4	Evolutibilidade / portabilidade	Trocar o mock por hardware OBD2 real, ou a lista de posições por um mapa real, sem reescrever telas nem a camada de serviços.
Q5	Segurança (meta futura)	Ainda sem autenticação de usuário, mas os microsserviços já não são públicos (exigem chave) e o CORS aceita só a origem do frontend. Antes de dados reais, entra o controle de acesso por perfil (RF14).
Q6	Implantação e evolução independentes	Cada microfrontend e cada microsserviço é compilado, testado e publicado

#	Objetivo de qualidade	Motivação (dirigida pelas personas)
		separadamente; uma falha em um não derruba os outros.

1.3. Partes Interessadas

Papel / Nome	Contato	Expectativa em relação à arquitetura
Motorista — Juliana Ferreira (persona)	Usuário final (campo)	App simples, utilizável com internet ruim; registro rápido de POD.
Coordenador de Frota — Marcos Andrade (persona)	Usuário final (escritório)	Dashboard responsivo com mapa, lista de pedidos por entrega e alertas confiáveis.
Diretor de Logística — Roberto Lima (persona)	Usuário final (esporádico)	Relatórios/KPIs consolidados e corretos; baixo custo operacional.
Equipe de desenvolvimento	Grupo PJBL (4 integrantes)	Arquitetura que permita dividir o trabalho por microfrontend e por microsserviço, com regras de dependência verificadas por testes; deploy automatizado e independente.
Professor / Avaliador	Disciplina PUC	Aderência a Arc42, C4 e UML com notação correta; rastreabilidade requisito ↔ arquitetura.
Transportadora (cliente futuro)	—	Confiabilidade, segurança dos dados e custo previsível em produção.

1.4. Software Architecture Canvas



SOFTWARE ARCHITECTURE CANVAS

Plataforma de Telemetria e Gestão de Entregas

Arquitetura de Soluções · PUC
Grupo: Fernando Padilha · Hector Saldivar · Theo Otto · Vinicius Veiga
Entrega parcial — documentação Arc42 · set/2026

01 NOME E PROPÓSITO

Plataforma de Telemetria e Gestão de Entregas. Dá à transportadora visibilidade da frota e das entregas em tempo real e reduz falhas e atrasos na operação logística.

02 CONTEXTO DE NEGÓCIO

Transportadora com frota própria: motoristas em campo, coordenação no escritório e diretoria acompanhando indicadores. Hoje o mercado resolve isso com planilhas e rastreadores isolados, sem gestão de pedidos integrada.

03 STAKEHOLDERS-CHAVE

- **Motorista** — entregas do dia, prova de entrega (POD)
- **Coordenador de Frota** — mapa, pedidos por entrega, alertas
- **Diretor de Logística** — KPIs consolidados (custo, falhas, prazos)
- **Equipe de desenvolvimento** — grupo PJBL (4 integrantes)
- **Professor / avaliador** — aderência a Arc42, C4 e UML

04 REQUISITOS-CHAVE

Funcionais. CRUD de pedidos por entrega (RF05 / RF07): **implementado**. Telemetria OBD2/GPS e modo offline (RF01–RF04); alertas de velocidade, falha mecânica e desvio de rota (RF08–RF10); POD (RF11); relatórios e KPIs (RF13); acesso por perfil (RF14): planejados.

Qualidade. Disponibilidade com internet instável · custo zero (free tier) · manutenibilidade · evolutibilidade.

05 RESTRIÇÕES

Planos gratuitos (Azure Static Web Apps Free, Azure Functions no plano de Consumo, MongoDB Atlas M0). Backend 100% serverless em Azure Functions Node.js 22; frontend React/Vite com Module Federation. Sem autenticação de usuário nesta fase. Diagramas em PlantUML / C4.

06 BLOCOS PRINCIPAIS

Shell, mfe-pedidos e mfe-frota (Static Web Apps) · API Gateway · BFF Web · Serviço de Pedidos · Serviço de Frota · filas `pedidos-eventos` e `telemetria-veiculos` · bancos `pjbl_pedidos` e `pjbl_frota`.

Planejados: App do Motorista e BFF Mobile.

07 PRINCÍPIOS E DECISÕES-CHAVE

Microfrontends (shell + remotos); API Gateway como única porta pública; BFF por tipo de cliente; microsserviços com Database per Service; colaboração entre serviços por eventos (Azure Queue Storage); Clean Architecture + Vertical Slice em todo backend; regras de arquitetura verificadas por testes no CI. ADRs detalhados na seção 9 do Arc42.

08 DADOS

Pedido (banco `pjbl_pedidos`, dono: Serviço de Pedidos); Veículo e Entrega (banco `pjbl_frota`, dono: Serviço de Frota). A entrega é uma projeção mantida pelos eventos de pedido. Ver DER, seção 8.

09 ROADMAP (PLANEJADO)

1. Autenticação por perfil (RF14)
2. Ingestão de telemetria OBD2/GPS + modo offline
3. App mobile do motorista + POD
4. Alertas e relatórios gerenciais
5. Mapa da frota em tempo real

10 RISCOS E DÍVIDA TÉCNICA

- Sem autenticação de usuário.
- Publicação de eventos sem **outbox**.
- Consistência eventual entre serviços.
- **Cold start** em cadeia (gateway → BFF → serviço).
- Rate limit por instância.

Bloco	Conteúdo
Nome e propósito	Plataforma de Telemetria e Gestão de Entregas — dar visibilidade da frota/entregas em tempo real e reduzir falhas e atrasos.
Contexto de negócio	Transportadora com frota própria; motoristas em campo, coordenação no escritório, diretoria acompanhando indicadores. Concorrentes: planilhas e rastreadores isolados sem gestão de pedidos.
Stakeholders-chave	Motorista (POD, entregas do dia), Coordenador (mapa, pedidos, alertas), Diretor (KPIs), Equipe de dev, Professor/avaliador.
Requisitos-chave	Funcionais: CRUD de pedidos por entrega (RF05/RF07 — feito), telemetria OBD2/GPS + offline (RF01–RF04), alertas (RF08–RF10), POD (RF11), relatórios (RF13), perfis (RF14). Qualidade: disponibilidade com internet instável, custo zero, manutenibilidade, evolutibilidade.

Bloco	Conteúdo
Restrições	Planos gratuitos (Azure SWA Free, Functions plano de Consumo, Atlas M0); backend 100% serverless em Azure Functions Node.js 22; frontend React/Vite com Module Federation; sem autenticação de usuário nesta fase; diagramas em PlantUML/C4.
Princípios e decisões-chave	Microfrontends (shell + remotos); API Gateway como única porta pública; BFF por tipo de cliente; microsserviços com Database per Service; colaboração entre serviços por eventos (Azure Queue Storage); Clean Architecture + Vertical Slice em todo backend; regras de arquitetura verificadas por testes no CI. Ver ADRs na seção 9.
Blocos principais	Shell, mfe-pedidos e mfe-frota (Static Web Apps) · API Gateway · BFF Web · Serviço de Pedidos · Serviço de Frota · filas pedidos-eventos e telemetria-veiculos · bancos pjbl_pedidos e pjbl_frota. Planejados: App do Motorista e BFF Mobile.
Dados	Pedido (banco pjbl_pedidos, dono: Serviço de Pedidos); Veículo e Entrega (banco pjbl_frota, dono: Serviço de Frota). A entrega é uma projeção mantida pelos eventos de pedido. Ver DER, seção 8.
Riscos e dívida	Sem autenticação de usuário; publicação de eventos sem *outbox*; consistência eventual entre serviços; *cold start* em cadeia (gateway → BFF → serviço); rate limit por instância. Ver seção 11.
Roadmap (planejado)	1) Autenticação por perfil (RF14). 2) Ingestão de telemetria OBD2/GPS + offline. 3) App mobile + POD. 4) Alertas e relatórios. 5) Mapa real.

2. Restrições da Arquitetura

2.1. Restrições Técnicas

#	Restrição	Consequência para a arquitetura
TC01	Hospedagem sem servidor dedicado — Azure Static Web Apps (Free) para o frontend e Azure Functions no plano de Consumo para o backend	Frontend estático; cada backend é um Function App próprio, com escala automática e cobrança por execução.
TC02	Backend em Azure Functions, Node.js 22, modelo de programação v4, em 4 Function Apps independentes (gateway, BFF, pedidos, frota)	Funções registradas por `app.http()` (HTTP) e `app.storageQueue()` (fila); cada projeto tem `package.json` e `host.json` próprios.
TC03	MongoDB Atlas cluster M0 com um banco por microsserviço (`pjbl_pedidos`, `pjbl_frota`) e um usuário por serviço	~512 MB no total; sem Private Endpoint no free tier; isolamento garantido pelas permissões dos usuários.
TC04	Frontend SPA React 19 + Vite 8	Variáveis `VITE_*` são resolvidas em tempo de build e embutidas no bundle — não servem para segredos.
TC05	Sem autenticação de usuário nesta fase	Só o API Gateway é anônimo; BFF e microsserviços exigem a chave da Function (`authLevel: 'function'`); CORS restrito às origens do frontend.
TC06	Diagramas em PlantUML + C4-PlantUML (ferramenta gratuita, local)	Notação C4 "oficial" (níveis 1–3) e UML/DER em PlantUML puro (nível 4 e DER).
TC07	Mensageria com Azure Queue Storage	Entrega "pelo menos uma vez", sem ordem garantida: os consumidores precisam ser idempotentes e tolerar eventos fora de ordem.
TC08	Microfrontends com Module Federation (`@module-federation/vite`)	Shell e remotos compartilham React como singleton; remotos são publicados em subpastas do mesmo Static Web App.

2.2. Restrições Organizacionais

#	Restrição

#	Restrição
OC01	Trabalho em grupo (4 integrantes), com prazos da disciplina; entrega parcial (princípios de arquitetura cloud ainda não são totalmente avaliados).
OC02	Uso de IA generativa é permitido e deve ser registrado (`Prompt.md` / `docs/arc42/Prompt-IAG.md`).
OC03	Repositório único no GitHub; deploy contínuo a cada `push` na branch `main` via GitHub Actions.
OC04	As atividades anteriores (microfrontend + Function mock; CRUD com MongoDB; refatoração para Vertical Slice + Clean Architecture) já foram entregues. Esta etapa evolui o mesmo sistema para microsserviços, microfrontends e eventos, mantendo o contrato das telas.

2.3. Convenções

#	Convenção
CV01	Código, comentários e textos de documentação em português.
CV02	Identificadores em português; sintaxe de bibliotecas mantida no idioma original (`Person()`, `System()`, `app.http()`).
CV03	Não introduzir dependências sem ganho didático/arquitetural claro (ex.: navegação sem `react-router` para 2 telas).
CV04	Títulos de diagramas em português; um arquivo `.puml` por diagrama em `docs/arc42/diagramas/`.
CV05	Todo backend segue a mesma estrutura: `dominio/`, `aplicacao/`, `features/<área>/<fatia>/` (com `*Function.js`, `*CasoDeUso.js`, `Porta*` e `Comando`/`Filtro`), `infraestrutura/` e `composicao/`. As regras de dependência

#	Convenção
	entre essas pastas são verificadas pelos testes de arquitetura.

3. Contexto e Escopo

3.1. Contexto de Negócio

O sistema é tratado como caixa-preta; a tabela lista os parceiros externos e o que entra/sai em termos de domínio.

Parceiro externo	Entradas para o sistema	Saídas do sistema
Motorista (Juliana)	Identificação (futuro), confirmação de entrega e POD	Lista de entregas do dia, rota, situação de cada pedido
Coordenador de Frota (Marcos)	Cadastro/edição/exclusão de pedidos; filtros de pesquisa; (futuro) parâmetros de alerta	Tabela de pedidos por entrega, posição da frota, alertas
Diretor de Logística (Roberto)	Período e filtros de relatório	KPIs consolidados (custo, falhas, prazos)
Dispositivo OBD2/GPS do veículo (simulado)	Leituras de telemetria: placa, posição GPS e velocidade	— (comunicação assíncrona por fila, sem resposta)

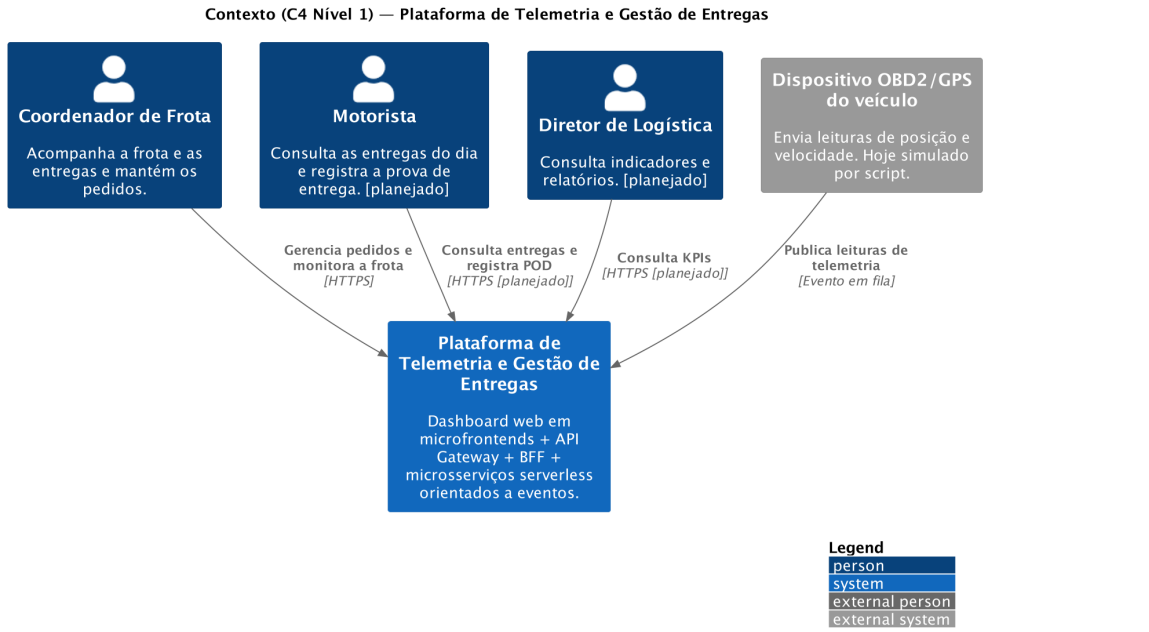


Figura 1 — Diagrama de Contexto (C4 Nível 1). Fonte: `diagramas/c4-nivel1-contexto.puml`.

3.2. Contexto Técnico

Canal	Tecnologia / Protocolo	Entrada/Saída transportada
Navegador ↔ Azure Static Web Apps	HTTPS	Shell (HTML/JS/CSS) e `remoteEntry.js` de cada microfrontend
Microfrontends ↔ API Gateway	HTTPS + JSON, `/api/web/*`	Todas as requisições de dados das telas
API Gateway ↔ BFF Web	HTTPS + `x-functions-key` + `x-correlation-id`	Requisições encaminhadas
BFF Web ↔ Serviço de Pedidos / Serviço de Frota	HTTPS + `x-functions-key`	CRUD de pedidos; veículos e entregas
Serviço de Pedidos → fila `pedidos-eventos`	Azure Queue Storage (HTTPS)	Eventos PedidoCadastrado / Alterado / Excluido
Dispositivo OBD2/GPS → fila `telemetria-veiculos`	Azure Queue Storage (HTTPS)	Evento LeituraDeTelemetriaRecebida
Filas → Serviço de Frota	Queue Trigger do Azure Functions	Mesmos eventos, consumidos de forma assíncrona
Microsserviços ↔ MongoDB Atlas	MongoDB Wire Protocol sobre TLS	Documentos de cada banco (um banco por serviço)
GitHub ↔ Azure	HTTPS (GitHub Actions)	Build e deploy do frontend e dos 4 Function Apps
App mobile ↔ API Gateway (planejado)	HTTPS/JSON, `/api/mobile/*` → BFF Mobile	Entregas do dia + POD

As entradas e saídas do Coordenador trafegam pelos canais Navegador ↔ Static Web Apps e Microfrontends ↔ API Gateway; a telemetria do veículo entra pela fila `telemetria-veiculos`. Nenhum microsserviço é acessível diretamente pelo navegador.

4. Estratégia de Solução

Objetivo / Requisito dirigente	Abordagem arquitetural	Detalhe na solução
Custo zero (Q2)	Serverless + planos gratuitos	4 Function Apps no plano de Consumo + Static Web Apps Free + Atlas M0 + Queue Storage
Evolução independente das telas (Q6)	Microfrontends com Module Federation	Shell (host) carrega `mfe-pedidos` e `mfe-frota` em tempo de execução; cada um com build próprio
Uma única porta de entrada (Q5)	API Gateway	Function anônima com tabela de rotas, CORS restrito, rate limit e correlation-id
Telas simples, sem lógica de integração (Q3)	Backend for Frontend	BFF Web junta pedidos + situação do veículo e monta o painel da frota numa só chamada
Escalar e implantar partes separadamente (Q6)	Microsserviços	Serviço de Pedidos e Serviço de Frota, cada um num Function App
Autonomia de dados (Q4)	Database per Service	`pjbl_pedidos` e `pjbl_frota`, com usuário próprio por serviço
Desacoplamento temporal entre serviços (Q1, Q4)	Arquitetura Orientada a Eventos	Pedidos publica eventos; Frota consome e mantém a sua projeção de entregas; telemetria entra por fila
Regras de negócio protegidas de frameworks (Q3, Q4)	Clean Architecture + Vertical Slice	Domínio sem bibliotecas; uma fatia por caso de uso; Composition Root injeta os adaptadores
Arquitetura que não se degrada com o tempo (Q3)	Testes Unitários de Arquitetura	65 regras verificadas no código a cada push, antes de qualquer deploy
Disponibilidade com rede instável (Q1)	Degradação graciosa em camadas	MFE com modo offline visível; BFF devolve

Objetivo / Requisito dirigente	Abordagem arquitetural	Detalhe na solução
		pedidos mesmo sem o serviço de frota; filas com retentativa

5. Visão de Blocos de Construção

5.1. Nível 1 — Sistema (caixa-branca) = C4 Nível 2 (Contêineres)

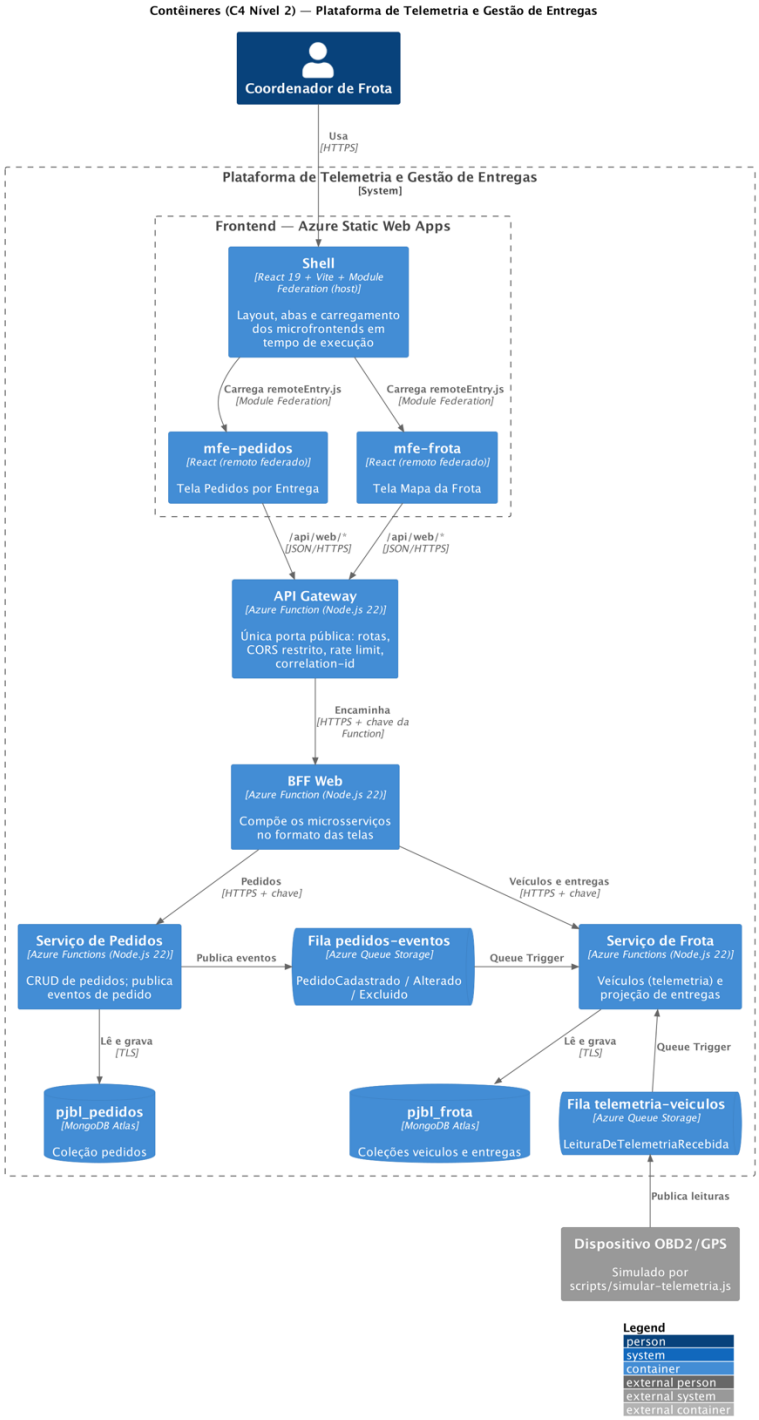


Figura 2 — Diagrama de Contêineres (C4 Nível 2). Fonte: `diagramas/c4-nivel2-conteineres.puml`.

Motivação da decomposição: o frontend é dividido por tela em microfrontends independentes, integrados pelo shell; o acesso ao backend passa sempre pelo API Gateway e pelo BFF, que isolam as telas da forma como os serviços estão organizados; o domínio é dividido em dois microsserviços — Pedidos e Frota — com bancos separados, que colaboram por eventos em vez de chamadas diretas.

Blocos contidos (caixa-preta):

Bloco	Responsabilidade
Shell (React 19 + Vite + Module Federation)	Layout, abas, carregamento dos microfrontends em tempo de execução e isolamento de falhas entre eles.
mfe-pedidos (remoto)	Tela Pedidos por Entrega: CRUD, pesquisa e conclusão de entrega.
mfe-frota (remoto)	Tela Mapa da Frota: indicadores, veículos e entregas, atualizada a cada 15 s.
API Gateway (Azure Function)	Única porta pública: roteamento por prefixo para o BFF, CORS restrito, rate limit (120 req/min por IP), correlation-id.
BFF Web (Azure Function)	Adapta os microsserviços às telas: pedidos com a situação do veículo, painel da frota, conclusão de entrega. Sem banco.
Serviço de Pedidos (Azure Functions)	CRUD de pedidos; dono do banco `pjbl_pedidos`; publica eventos de pedido.
Serviço de Frota (Azure Functions)	Veículos (telemetria) e projeção de entregas; dono do banco `pjbl_frota`; consome as filas.
Filas `pedidos-eventos` e `telemetria-veiculos` (Azure Queue Storage)	Transporte assíncrono dos eventos.
Bancos `pjbl_pedidos` e `pjbl_frota` (MongoDB Atlas)	Persistência, cada um acessível só pelo seu serviço.
App do Motorista + BFF Mobile (planejados)	Entregas do dia e POD, entrando pelo gateway em `/api/mobile/*`.

Interface importante — API pública (API Gateway):

Método	Rota pública	Atendida por	Sucesso	Erros

Método	Rota pública	Atendida por	Sucesso	Erros
GET	`/api/web/pedidos?cliente=&status=&entregaId=&id=`	BFF → Serviço de Pedidos + Serviço de Frota	200 [pedido + `veiculo`]	429, 502, 504
POST	`/api/web/pedidos`	BFF → Serviço de Pedidos	201 pedido	400, 429, 502
PUT	`/api/web/pedidos/{id}`	BFF → Serviço de Pedidos	200 pedido	400, 404, 429, 502
DELETE	`/api/web/pedidos/{id}`	BFF → Serviço de Pedidos	200 { `sucesso, id` }	404, 429, 502
POST	`/api/web/entregas/{id}/concluir`	BFF → Serviço de Pedidos (vários PUT)	200 { `entregaId, pedidosConcluidos` }	429, 502
GET	`/api/web/painel-frota`	BFF → Serviço de Frota	200 { `indicadores, veiculos, entregas` }	429, 502, 504

Toda resposta do gateway traz `x-correlation-id` e `x-ratelimit-remaining`; o CORS só é liberado para a origem do Static Web App. Rota desconhecida → 404; método não permitido → 405; limite excedido → 429 com `Retry-After`.

5.2. Nível 2 — Contêineres de backend (caixa-branca) = C4 Nível 3 (Componentes)

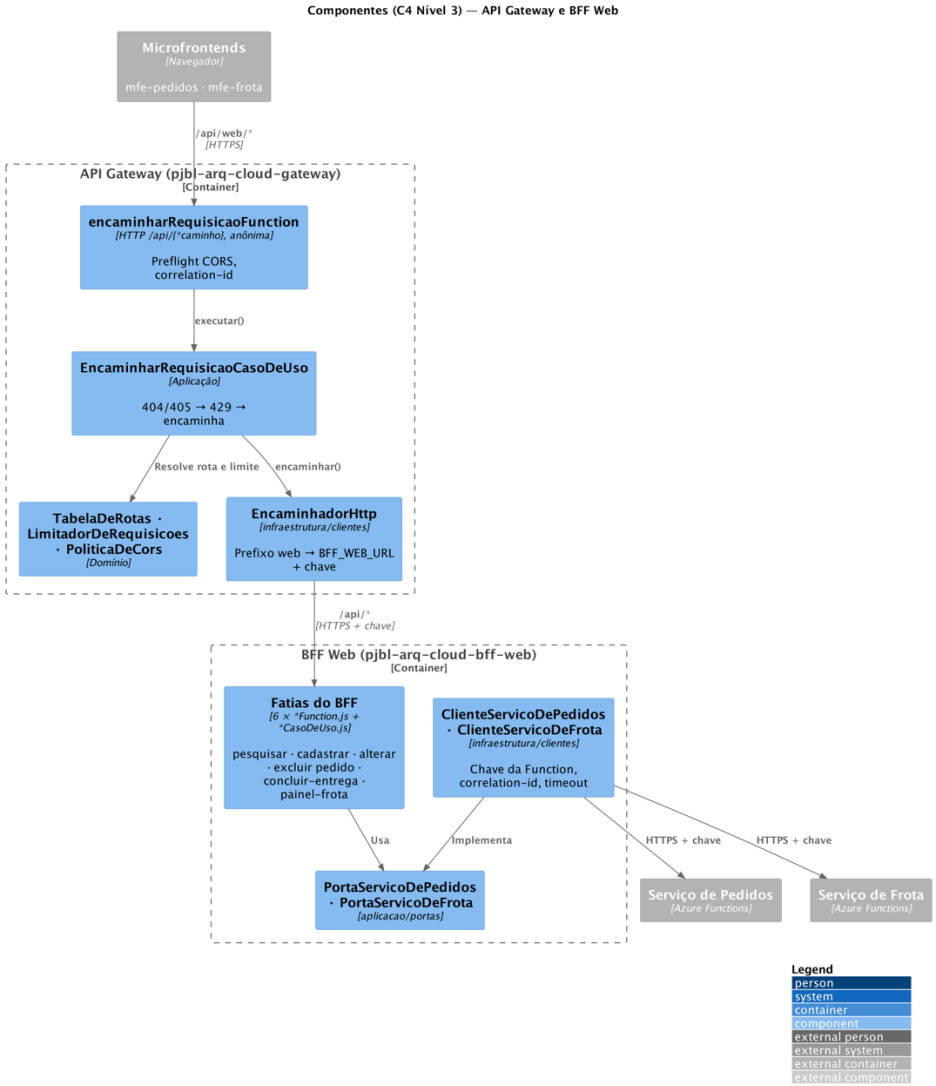


Figura 3 — Diagrama de Componentes (C4 Nível 3): API Gateway e BFF Web. Fonte: `diagramas/c4-nivel3-gateway-bff.puml`.

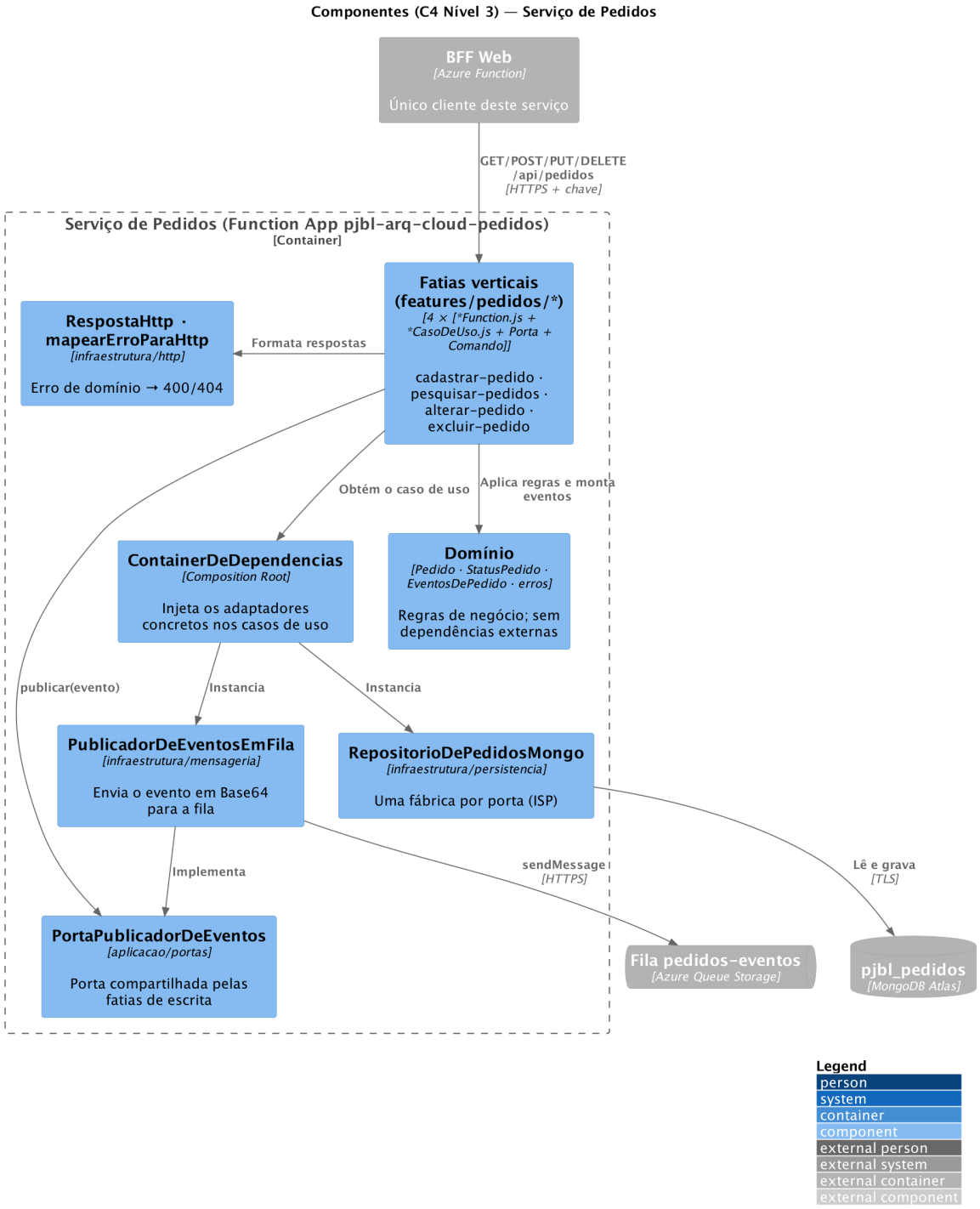


Figura 4 — Diagrama de Componentes (C4 Nível 3): Serviço de Pedidos. Fonte: `diagramas/c4-nivel3-servico-pedidos.puml`.

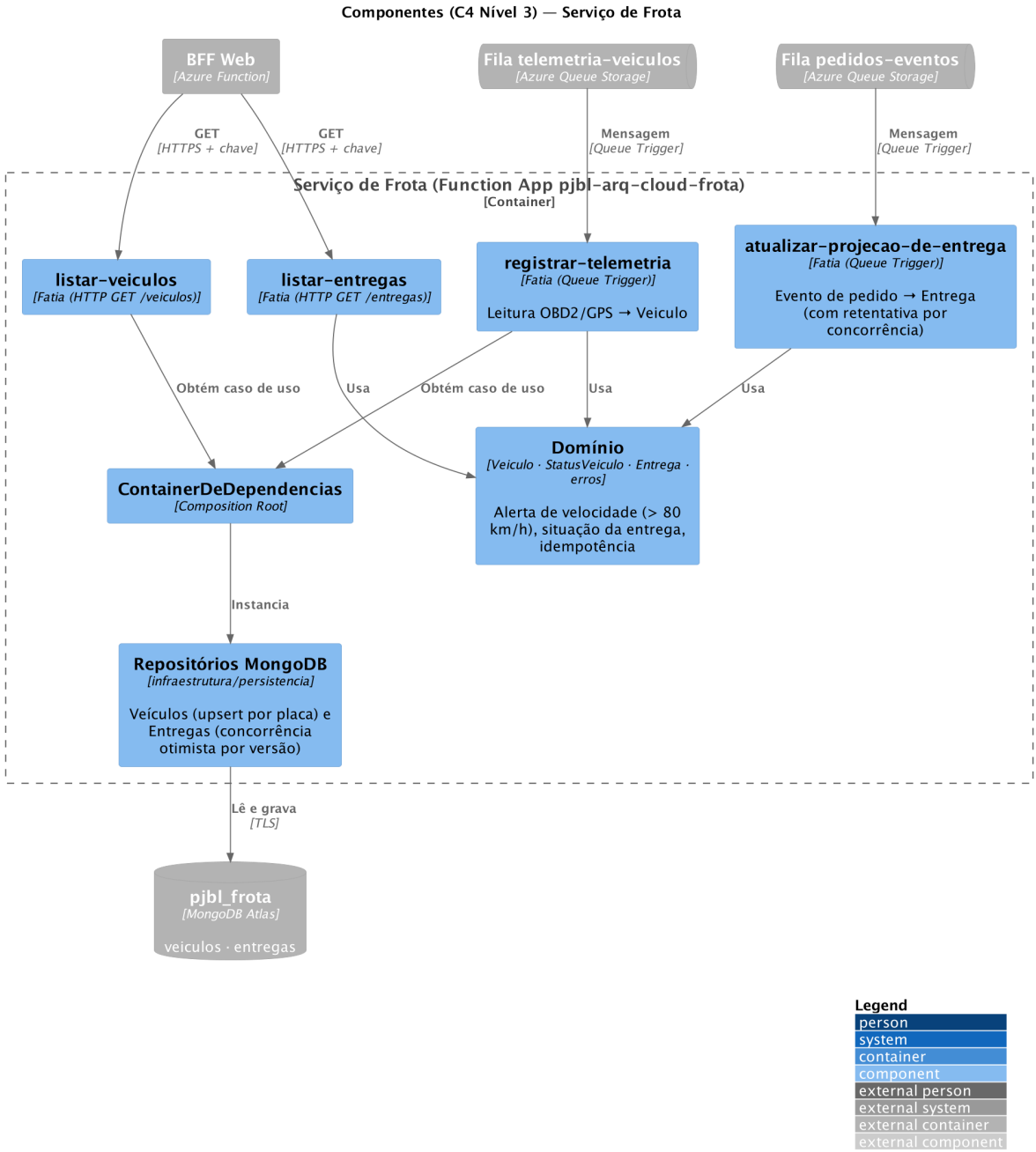


Figura 5 — Diagrama de Componentes (C4 Nível 3): Serviço de Frota. Fonte: `diagramas/c4-nivel3-servico-frota.puml`.

Contêiner	Fatia vertical (`features/...`)	Entrada	Responsabilidade
API Gateway	`roteamento/encaminhar-requisicao`	HTTP `/api/{*caminho}` (anônima)	Resolve a rota, aplica limite e encaminha ao BFF com a chave

Contêiner	Fatia vertical (`features/…`)	Entrada	Responsabilidade
BFF Web	`pedidos/pesquisar-pedidos`	HTTP GET	Pedidos + situação do veículo (degrada sem o serviço de frota)
BFF Web	`pedidos/cadastrar-pedido`, `alterar-pedido`, `excluir-pedido`	HTTP POST/PUT/DELETE	Repassa ao Serviço de Pedidos
BFF Web	`pedidos/concluir-entrega`	HTTP POST	Busca os pedidos da entrega e marca os pendentes como Entregue
BFF Web	`painel/obter-painel-frota`	HTTP GET	Veículos + entregas + indicadores
Serviço de Pedidos	`pedidos/cadastrar-pedido`, `alterar-pedido`, `excluir-pedido`	HTTP	Regra de negócio no domínio, grava e publica o evento
Serviço de Pedidos	`pedidos/pesquisar-pedidos`	HTTP GET	Consulta com filtros
Serviço de Frota	`veiculos/registrar-telemetria`	Fila `telemetria-veiculos`	Calcula o status do veículo e grava a última leitura
Serviço de Frota	`entregas/atualizar-projecao-de-entrega`	Fila `pedidos-eventos`	Aplica o evento à entrega (idempotente, concorrência otimista)
Serviço de Frota	`veiculos/listar-veiculos`, `entregas/listar-entregas`	HTTP GET	Leituras para o BFF

Cada fatia contém o seu adaptador de entrada (`*Function.js`), o caso de uso (`*CasoDeUso.js`), a porta (interface) de que precisa e o comando/filtro de entrada. As fatias não se conhecem entre si; o que elas compartilham fica em `dominio/` e `aplicacao/`, e só o Composition Root (`composicao/ContainerDeDependencias.js`) conhece as implementações de MongoDB, fila e HTTP.

5.3. Nível 3 — Detalhes

5.3.1. Modelo de classes do domínio (UML — Classes)

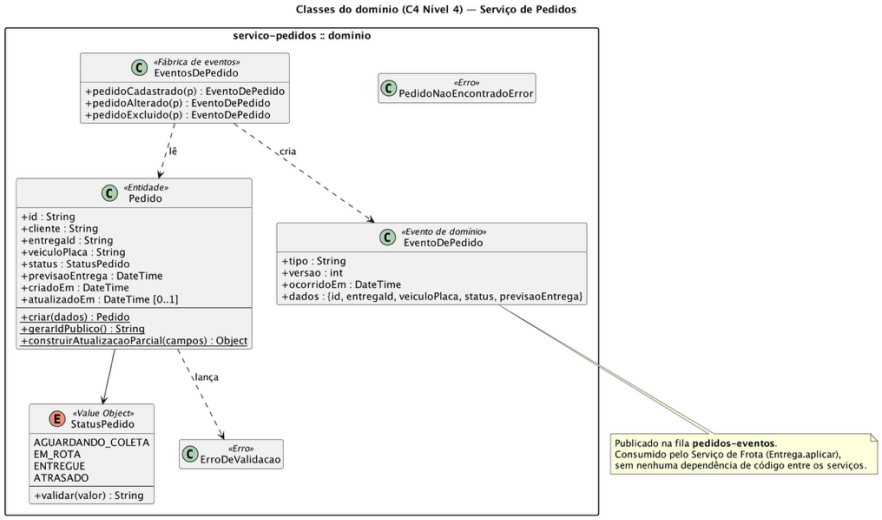


Figura 6 — Diagrama de Classes (C4 Nível 4): domínio do Serviço de Pedidos. Fonte: `diagramas/uml-classes-servico-pedidos.puml`.

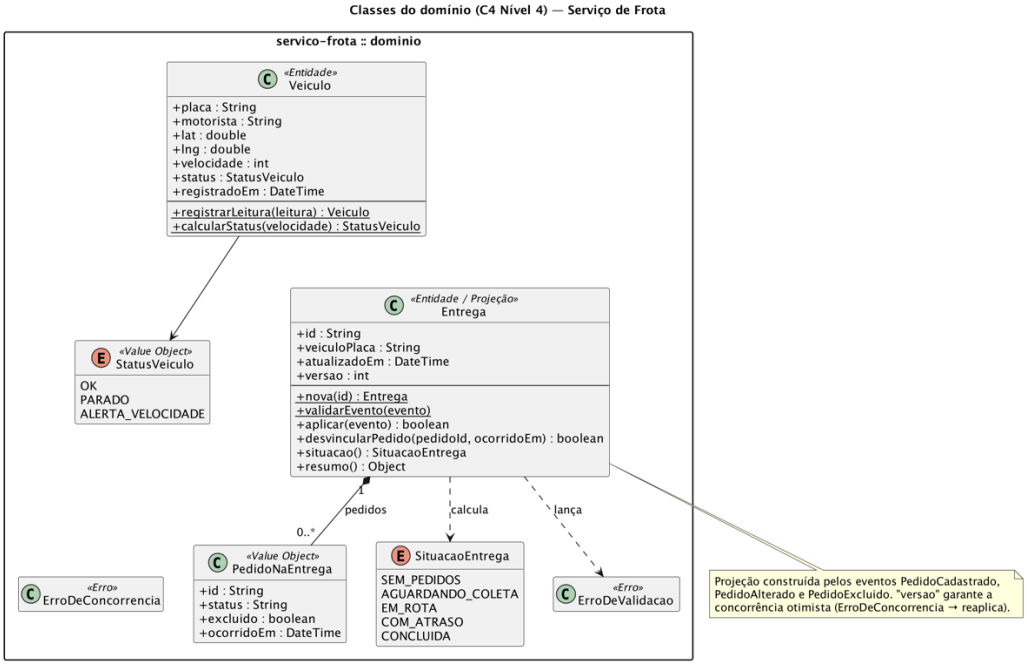


Figura 7 — Diagrama de Classes (C4 Nível 4): domínio do Serviço de Frota. Fonte: `diagramas/uml-classes-servico-frota.puml`.

O domínio está dividido entre os dois microsserviços e não há associação entre classes de serviços diferentes: a única ligação é o contrato do evento (`EventoDePedido`), publicado pelo Serviço de Pedidos e interpretado pelo Serviço de Frota em `Entrega.aplicar()`.

No Serviço de Pedidos, `Pedido` concentra as regras de cadastro e de atualização parcial, `StatusPedido` é o conjunto fechado de status e `EventosDePedido` monta os eventos com envelope versionado (`tipo`, `versao`, `ocorridoEm`, `dados`). No Serviço de Frota, `Veiculo` calcula o status a partir da velocidade (alerta acima de 80 km/h — RF08) e `Entrega` é uma projeção que guarda, para cada pedido, o último estado conhecido e a data do último evento aplicado, ignorando eventos antigos e mantendo "lápides" de pedidos excluídos. O campo `versao` sustenta o controle de concorrência otimista.

5.3.2. Componentes do frontend em microfrontends

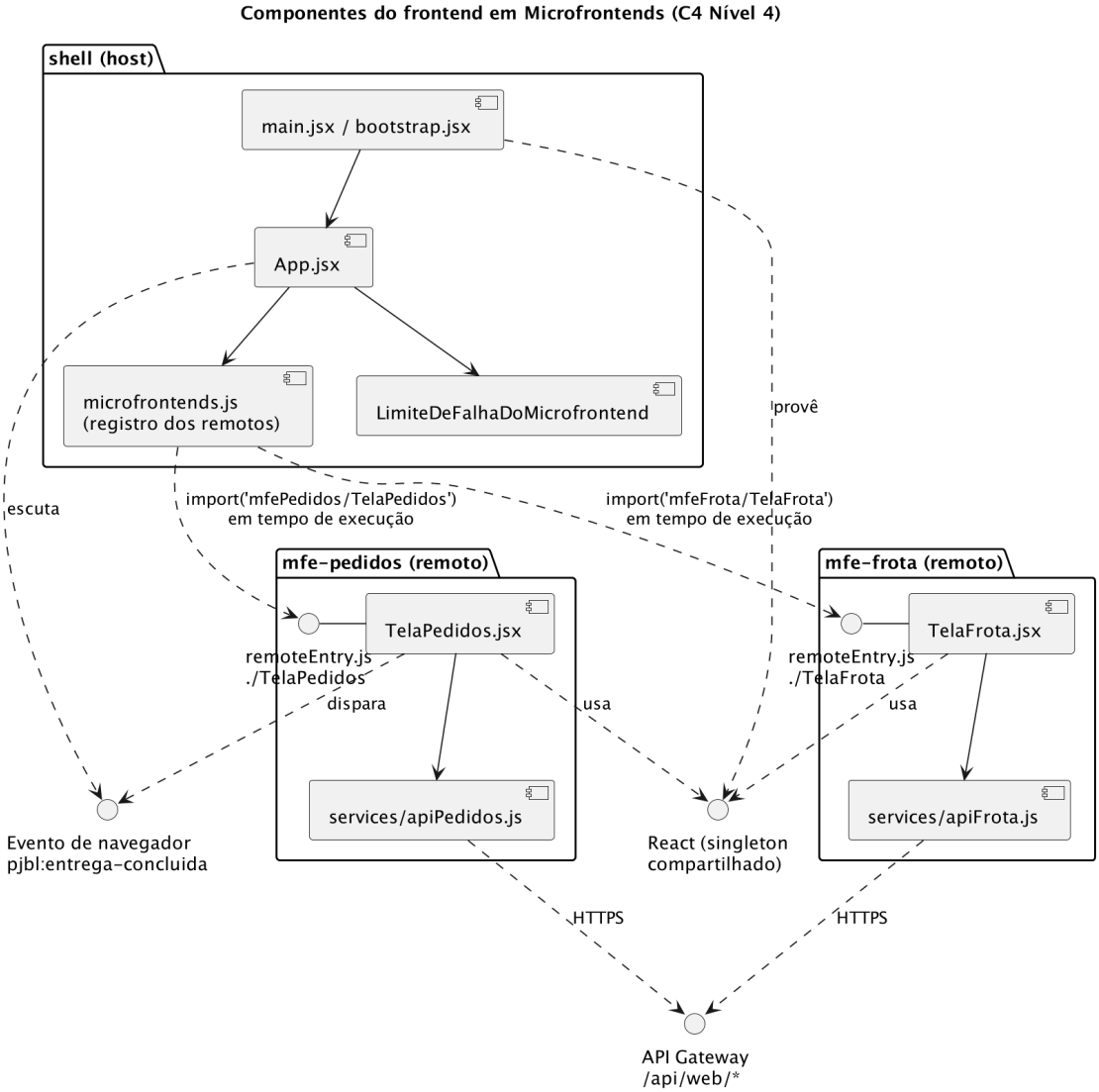


Figura 8 — Diagrama de Componentes (C4 Nível 4): módulos internos dos microfrontends e suas interfaces. Fonte: `diagramas/uml-componentes-frontend.puml`.

O shell (host) contém apenas o layout, o registro dos microfrontends (`microfrontends.js`) e um limite de falha por microfrontend. Cada remoto expõe uma tela pelo seu `remoteEntry.js` (`mfePedidos/TelaPedidos`, `mfeFrota/TelaFrota`), que o shell importa em tempo de execução. Cada microfrontend tem a sua própria camada `services/`, a única que faz chamadas HTTP, sempre para o API Gateway. React é

compartilhado como singleton. A comunicação entre microfrontends usa um evento de navegador combinado (`pjbl:entrega-concluida`), sem importação de código.

5.4. Relação com o modelo de dados

Cada microsserviço tem o seu banco; as estruturas persistidas e a forma como os dados de pedido chegam ao banco de frota (por eventos) estão no DER da seção 8.1.

6. Visão de Tempo de Execução

6.1. Cenário 1 — Coordenador cadastra um pedido

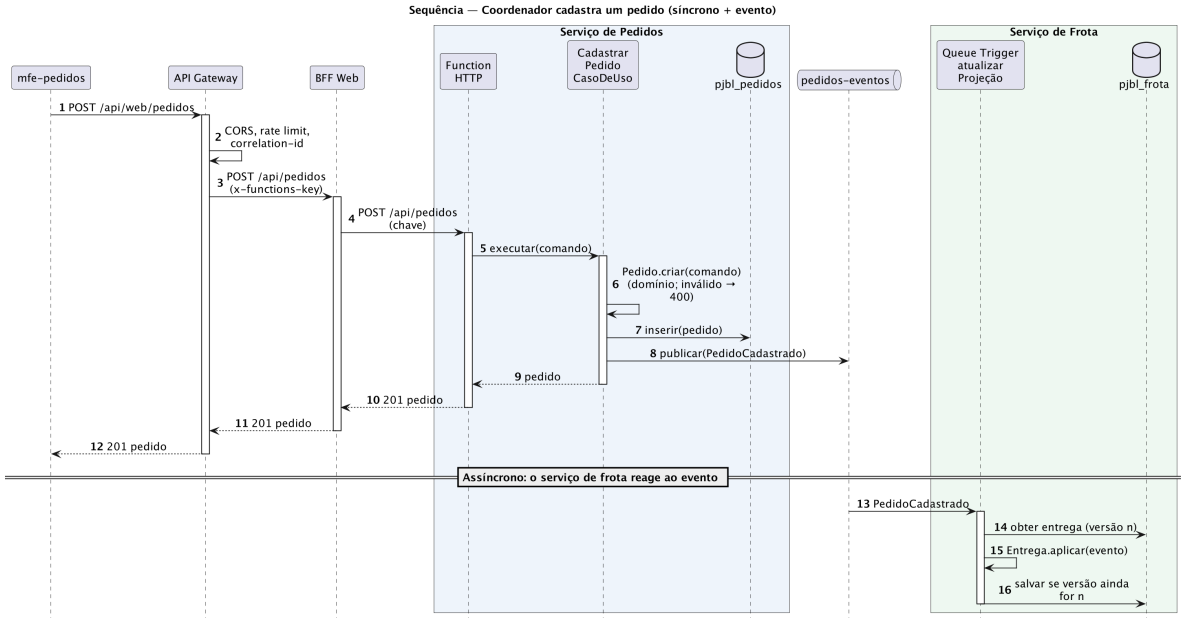


Figura 9 — Diagrama de Sequência (C4 Nível 4): cadastro de um pedido (síncrono + evento). Fonte: `diagramas/uml-sequencia-cadastrar-pedido.puml`.

Aspectos notáveis:

- A requisição atravessa três Function Apps: Gateway (CORS, limite, correlation-id) → BFF → Serviço de Pedidos. O mesmo `x-correlation-id` aparece nos logs dos três.
- A validação de negócio acontece no domínio (`Pedido.criar`); um erro de validação vira 400 no serviço e é repassado sem alteração pelo BFF e pelo gateway até a tela.
- Depois de gravar, o caso de uso publica `PedidoCadastrado` na fila. A resposta 201 não espera o Serviço de Frota: a atualização da entrega acontece de forma assíncrona (consistência eventual).
- A publicação é de "melhor esforço": se a fila falhar, o pedido já foi gravado e a falha fica registrada em log (ver risco R11).

6.2. Cenário 2 — Concluir entrega (eventos em paralelo)

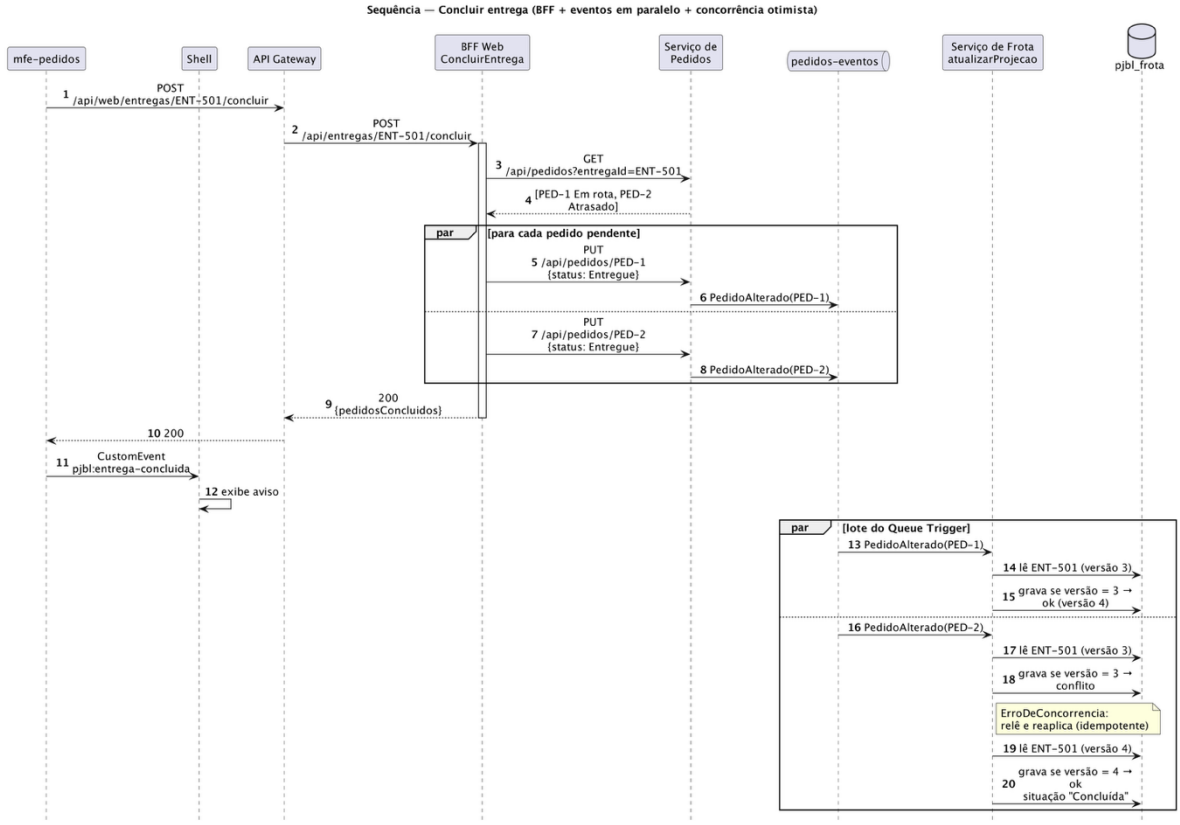


Figura 10 — Diagrama de Sequência (C4 Nível 4): concluir entrega (BFF + eventos em paralelo + concorrência otimista). Fonte: `diagramas/uml-sequencia-concluir-entrega.puml`.

Aspectos notáveis:

- A tela fala em "entrega"; o Serviço de Pedidos só conhece "pedido". O BFF faz a tradução: busca os pedidos da entrega e envia, em paralelo, um PUT para cada pedido pendente.
- Cada PUT gera um `PedidoAlterado`. O Queue Trigger processa as mensagens em lote, então dois eventos da mesma entrega podem ser aplicados ao mesmo tempo.
- O repositório só grava a entrega se a versão no banco ainda for a lida; no conflito, o caso de uso relê e reaplica o evento (até 5 tentativas). Sem isso, uma atualização se perderia e a entrega ficaria como "1 de 2 entregues" — situação reproduzida e corrigida durante o desenvolvimento.
- O mfe-pedidos dispara `pjbl:entrega-concluida`; o shell mostra o aviso sem conhecer o código do microfrontend.

6.3. Cenário 3 — Telemetria OBD2/GPS até o painel da frota

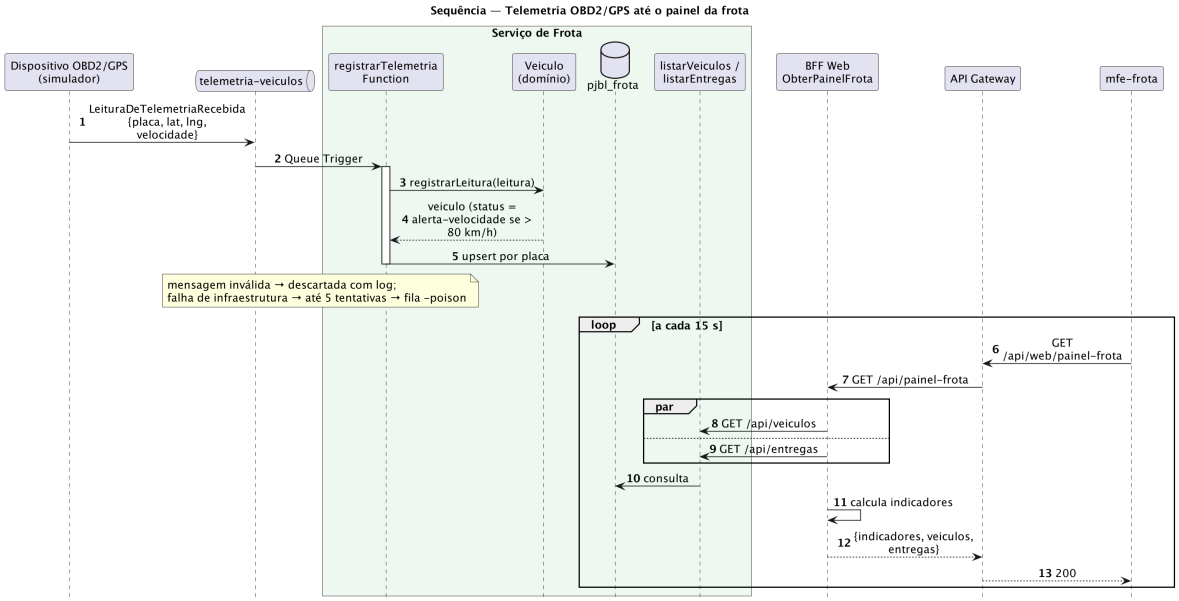


Figura 11 — Diagrama de Sequência (C4 Nível 4): telemetria OBD2/GPS até o painel da frota. Fonte: `diagramas/uml-sequencia-telemetria.puml`.

Aspectos notáveis:

- O dispositivo (hoje o simulador) só publica a leitura na fila e segue em frente; o Serviço de Frota processa no seu ritmo.
- O domínio `Veiculo` calcula o status (parado, normal ou excesso de velocidade). Mensagem inválida é descartada com log; falha de infraestrutura faz a mensagem voltar à fila — após 5 tentativas, vai para `telemetria-veiculos-poison`.
- O mfe-frota consulta o painel a cada 15 s; o BFF busca veículos e entregas em paralelo e calcula os indicadores.

6.4. Primeira invocação (cold start) e modo offline

Cada Function App mantém em cache, por instância, a conexão com o MongoDB (serviços) ou o cliente da fila, como na etapa anterior. Como uma requisição atravessa três Function Apps, o cold start pode se somar na primeira chamada após ociosidade (risco R10). Se o gateway não responder, cada microfrontend usa dados locais e exibe o aviso "Modo offline" — a indisponibilidade deixa de ser silenciosa.

7. Visão de Implantação

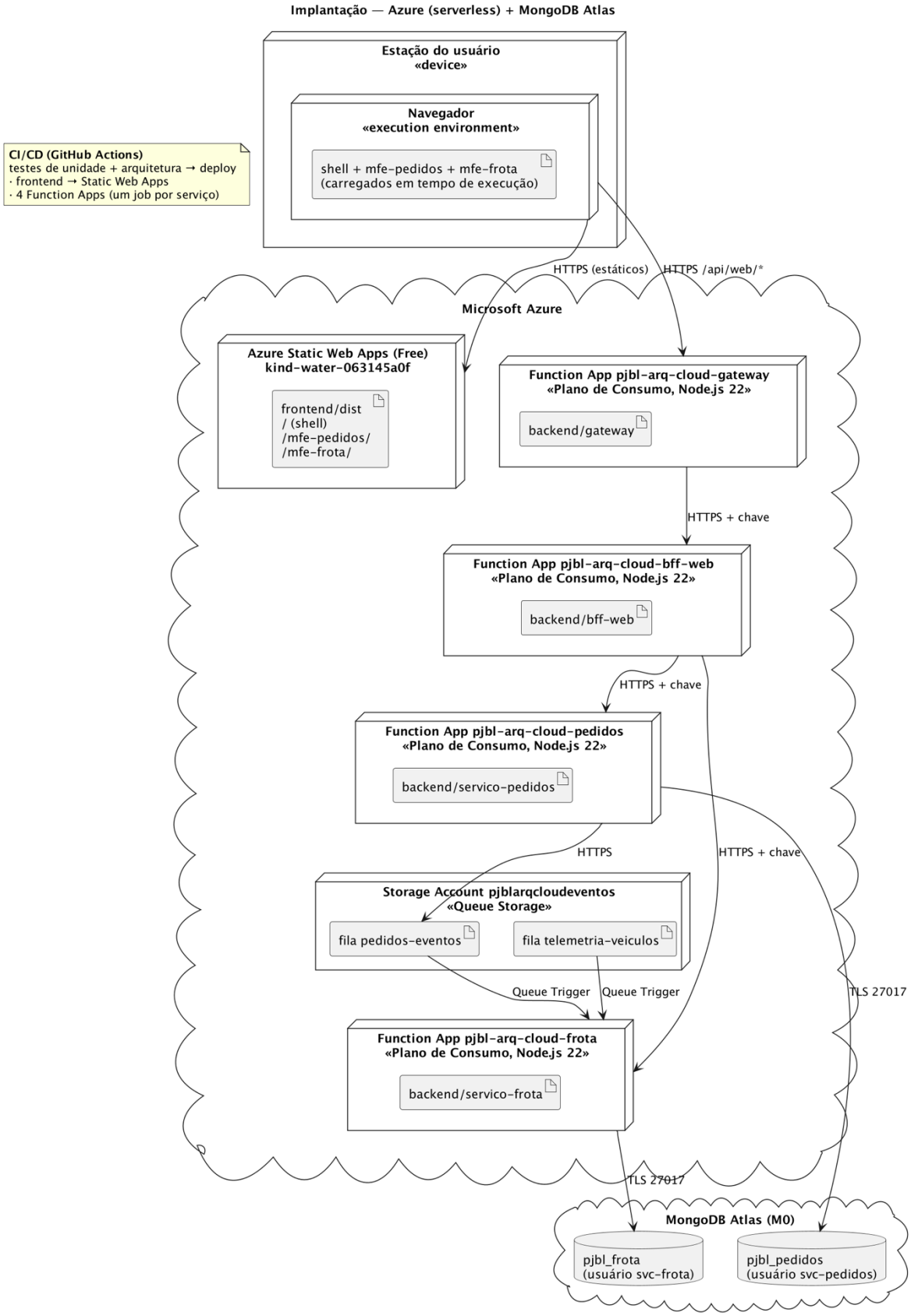

Implantação — Azure (serverless) + MongoDB Atlas
Estação do usuário
«device»
Navegador
«execution environment»
shell + mfe-pedidos + mfe-frota
(carregados em tempo de execução)
CI/CD (GitHub Actions)
testes de unidade + arquitetura → deploy
· frontend → Static Web Apps
· 4 Function Apps (um job por serviço)
HTTPS (estáticos)
HTTPS /api/web/*
Microsoft Azure
Azure Static Web Apps (Free)
kind-water-063145a0f
frontend/dist
/ (shell)
/mfe-pedidos/
/mfe-frota/
Function App pjbl-arq-cloud-gateway
«Plano de Consumo, Node.js 22»
backend/gateway
HTTPS + chave
Function App pjbl-arq-cloud-bff-web
«Plano de Consumo, Node.js 22»
backend/bff-web
HTTPS + chave
Function App pjbl-arq-cloud-pedidos
«Plano de Consumo, Node.js 22»
backend/servico-pedidos
HTTPS
HTTPS + chave
Storage Account pjblarqcloudeventos
«Queue Storage»
fila pedidos-eventos
fila telemetria-veiculos
Queue Trigger
Queue Trigger
TLS 27017
Function App pjbl-arq-cloud-frota
«Plano de Consumo, Node.js 22»
backend/servico-frota
TLS 27017
MongoDB Atlas (M0)
pjbl_frota
(usuário svc-frota)
pjbl_pedidos
(usuário svc-pedidos)

Figura 12 — Diagrama de Implantação (UML). Fonte: `diagramas/deployment.puml`.

Nó	Ambiente de execução	Artefato(s) implantado(s)
Estação do usuário	Navegador	Shell + microfrontends carregados em tempo de execução
Azure Static Web Apps (Free)	Hospedagem estática gerenciada	`frontend/dist`: shell na raiz, `/mfe-pedidos/`, `/mfe-frota/`
Function App `pjbl-arq-cloud-gateway`	Azure Functions, plano de Consumo, Node.js 22	`backend/gateway`
Function App `pjbl-arq-cloud-bff-web`	idem	`backend/bff-web`
Function App `pjbl-arq-cloud-pedidos`	idem	`backend/servico-pedidos`
Function App `pjbl-arq-cloud-frota`	idem	`backend/servico-frota`
Storage Account `pjblarqcloudeventos`	Azure Queue Storage	Filas `pedidos-eventos` e `telemetria-veiculos`
MongoDB Atlas (M0)	Cluster gerenciado	Bancos `pjbl_pedidos` e `pjbl_frota`, um usuário por serviço
GitHub Actions	Runner ubuntu-latest	Testes, build e deploy (não hospeda a aplicação)

Pipeline CI/CD (`.github/workflows/azure-static-web-apps-kind-water-063145a0f.yml`):

1. Push em qualquer branch ou PR: workflow Testes — 76 testes de unidade, 65 testes de arquitetura e build dos três microfrontends.
2. Push na `main` com mudanças em `backend/`: workflow Deploy do backend — roda os testes e, se passarem, publica os 4 Function Apps em paralelo (um job por serviço, `Azure/functions-action`).
3. Push na `main`: workflow Deploy do frontend — roda os testes, compila shell e remotos separadamente, monta `frontend/dist` e publica no Static Web App (`skip_app_build`).
4. Nenhum deploy acontece se uma regra de arquitetura for violada.

8. Conceitos Transversais

8.1. Modelo de dados

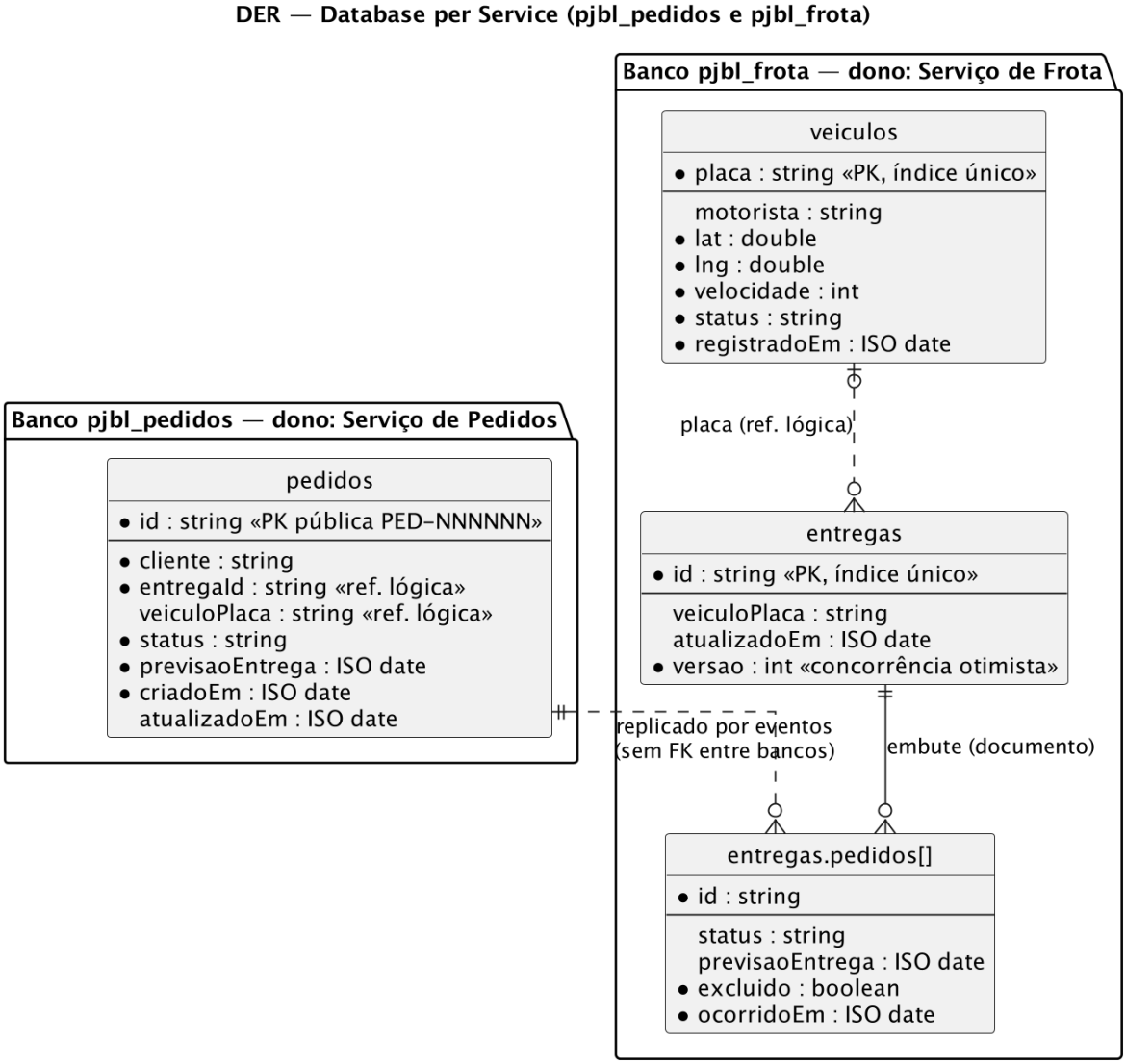


Figura 13 — Diagrama de Entidades e Relacionamentos (notação pé-de-galinha), um banco por serviço. Fonte: `diagramas/der-database-per-service.puml`.

O modelo é dividido por dono (Database per Service). O banco `pjbl_pedidos` tem a coleção `pedidos`, acessível só pelo Serviço de Pedidos. O banco `pjbl_frota` tem `veiculos` (última leitura por placa, índice único) e `entregas` (projeção com os pedidos embutidos e o campo `versao`), acessíveis só pelo Serviço de Frota.

Não existe chave estrangeira entre bancos. A relação pedido ↔ entrega é mantida por eventos: o Serviço de Frota guarda, para cada pedido, apenas o que precisa (id, status, previsão e data do último evento). O nome do cliente não é enviado nos eventos: o Serviço de Frota não precisa dele.

Documento `pedido`:

Campo	Tipo	Origem	Observação
`id`	string	domínio (Pedido.criar)	`PED-` + 6 dígitos; identificador público
`cliente`	string	cliente	obrigatório
`entregaId`	string	cliente	obrigatório; vínculo pedido ↔ entrega (RF07)
`veiculoPlaca`	string	cliente	opcional (`''` se ausente)
`status`	string	cliente	`Aguardando coleta` | `Em rota` | `Entregue` | `Atrasado`
`previsaoEntrega`	string (ISO)	cliente	default = momento da criação
`criadoEm`	string (ISO)	domínio (Pedido.criar)	definido em `createPedido`
`atualizadoEm`	string (ISO)	domínio (Pedido.construirAtualizacaoParcial)	definido em `updatePedido`
`_id`	ObjectId	driver MongoDB	interno; removido de todas as respostas

8.2. Configuração e segredos

Configuração exclusivamente por variáveis de ambiente, separadas por Function App: `MONGODB_URI`, `MONGODB_DB` e `EVENTOS_STORAGE` nos microsserviços; `SERVICO_PEDIDOS_URL/CHAVE` e `SERVICO_FROTA_URL/CHAVE` no BFF; `BFF_WEB_URL/CHAVE`, `ORIGENS_PERMITIDAS` e `LIMITE_REQUISICOES_POR_MINUTO` no gateway. Localmente vêm de `local.settings.json` (fora do Git); no Azure, dos Application settings. No frontend há uma única variável, `VITE_GATEWAY_URL`, pública e embutida no build (`.env.development` / `.env.production` de cada microfrontend).

8.3. Persistência em ambiente serverless

Cada microsserviço mantém o seu próprio cache da conexão com o MongoDB por instância do Function App. O Serviço de Frota cria, uma vez por instância, os índices únicos de `veiculos.placa` e `entregas.id`, que sustentam o upsert de veículos e o controle de concorrência das entregas.

8.4. Tratamento de erros e contrato de resposta

- Microsserviços: erro de domínio → 400/404 com `{ erro }`; falha de infraestrutura → 500.
- BFF: repassa 4xx do serviço sem alteração (erro do usuário); converte 5xx/indisponibilidade em 502 e timeout (8 s) em 504.
- Gateway: 404 (rota), 405 (método), 429 (limite), 502/504 (BFF indisponível ou lento).
- Consumidores de fila: mensagem inválida é descartada com log; falha de infraestrutura é relançada para a fila reentregar (até 5 vezes; depois, fila `-poison`).

8.5. CORS

O CORS é tratado somente no API Gateway, a única Function chamada pelo navegador. Em vez de `*`, são aceitas apenas as origens do Static Web App e dos servidores locais do Vite. BFF e microsserviços não enviam cabeçalhos CORS.

8.6. Autenticação e autorização

Ainda não há identidade de usuário (RF14 planejado). Já existe, porém, autenticação entre serviços: o gateway é anônimo, mas o BFF e os microsserviços usam `authLevel: 'function'` e só respondem a quem envia a chave (`x-functions-key`) — o gateway tem a chave do BFF e o BFF tem as dos serviços. O caminho previsto para RF14 é validar o token do usuário no gateway e repassar os papéis ao BFF.

8.7. Resiliência no cliente

Resiliência em camadas: (1) o shell isola falhas — se um microfrontend não carregar, só a área dele mostra o aviso; (2) cada microfrontend tem modo offline com aviso visível; (3) o BFF devolve os pedidos mesmo com o Serviço de Frota fora do ar (`veiculo: null`); (4) gateway e BFF usam timeout; (5) as filas desacoplam os serviços no tempo e reentregam mensagens; (6) o consumidor é idempotente e trata eventos fora de ordem e concorrentes.

8.8. Identificadores

O `id` público (`PED-NNNNNN`, string) é desacoplado do `_id` (ObjectId) do MongoDB. O frontend só conhece o `id`; o `_id` nunca sai da API.

8.9. Build e entrega contínua

Cada parte é compilada e publicada de forma independente: 3 builds de frontend e 4 deploys de Function App. Testes de unidade e de arquitetura são pré-requisito de todos os deploys (detalhes na seção 7).

8.10. Clean Architecture e Vertical Slice

Todos os backends seguem a mesma organização. A Regra de Dependência da Clean Architecture aponta sempre para dentro: adaptadores de entrada (`*Function.js`) → aplicação (casos de uso, portas, comandos) → domínio (sem bibliotecas). A infraestrutura (MongoDB, fila, clientes HTTP) implementa as portas e só é conhecida pelo Composition Root, que injeta os adaptadores nos casos de uso (Inversão de Dependência).

Por cima disso, o código é organizado em fatias verticais: uma pasta por caso de uso em `features/<área>/<fatia>/`, com tudo o que ele precisa. Uma fatia nunca importa outra; o que é compartilhado vive em `dominio/` e `aplicacao/`. Adicionar um caso de uso é criar uma fatia nova, sem tocar nas existentes.

8.11. Testes Unitários de Arquitetura

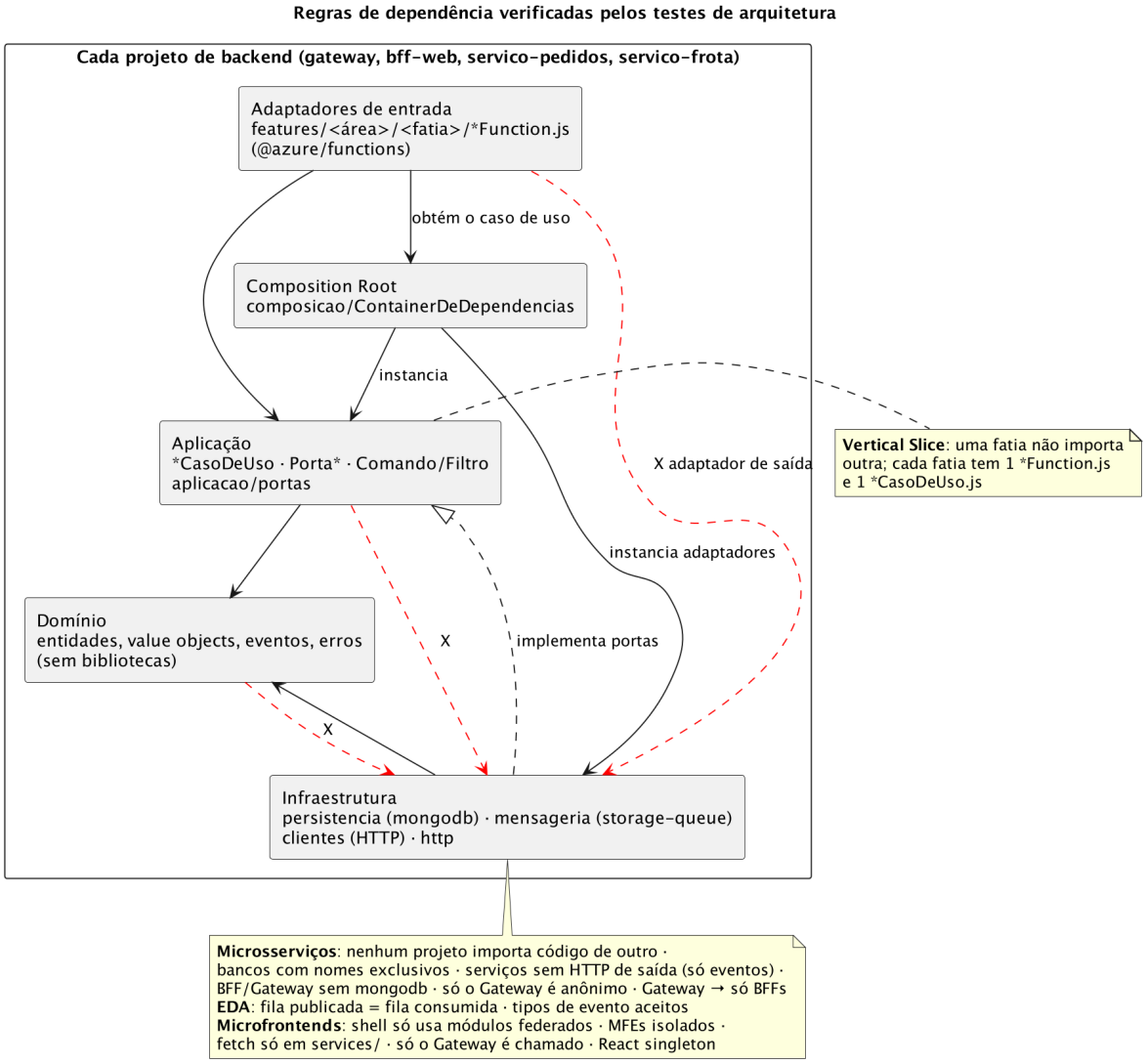


Figura 14 — Regras de dependência verificadas pelos testes de arquitetura. Fonte: `diagramas/regras-de-arquitetura.puml`.

As decisões de arquitetura são verificadas no código real por testes automatizados, no estilo do ArchUnit, escritos com o executor nativo do Node (`node:test`) e sem dependências externas (`testes-arquitetura/`). Um utilitário lê os `require`/`import` de cada arquivo e monta o grafo de dependências; cada regra percorre o grafo e falha listando as violações encontradas. São 65 verificações:

Arquivo	O que garante
`backend-clean-architecture.test.js`	Domínio sem bibliotecas nem outras camadas; aplicação só depende de aplicação e domínio; `mongodb`/`storage-queue` só na infraestrutura e `@azure/functions` só nos adaptadores de entrada; só o Composition Root conhece adaptadores de saída;

Arquivo	O que garante
	Functions não instanciam casos de uso
`backend-vertical-slice.test.js`	Fatias existentes = fatias documentadas; cada fatia tem 1 `*Function.js` e 1 `*CasoDeUso.js`; fatia não importa fatia
`backend-microsservicos-bff-gateway.test.js`	Nenhum projeto importa código de outro; bancos com nomes exclusivos e nunca citados por outro serviço; BFF e gateway sem banco; microsserviços sem HTTP de saída; gateway só roteia para BFFs; só o gateway é anônimo; todo ponto de entrada é uma Azure Function
`arquitetura-orientada-a-eventos.test.js`	Fila publicada = fila consumida; todo tipo de evento publicado é aceito pelo consumidor; envelope versionado; toda escrita publica evento e a leitura não; só a infraestrutura conhece a fila
`frontend-microfrontends.test.js`	Shell e remotos com build próprio e Module Federation; nenhum importa código de outro; shell usa só módulos federados; remotos registrados e expostos; React singleton; `fetch` só em `services/`; microfrontends só chamam o gateway; CSS com escopo

Para garantir que as regras realmente detectam problemas, foram injetadas violações de propósito (domínio importando `mongodb`, fatia importando fatia, serviço citando o banco do outro, serviço chamando outro por HTTP, Function instanciando caso de uso, microfrontend importando outro, tela chamando microsserviço direto) — cada uma foi apontada pela regra correspondente. Os testes rodam no CI antes de qualquer deploy.

8.12. Eventos

Envelope de todo evento: `{ id, origem, tipo, versao, ocorridoEm, dados }`. `versao` é a versão do contrato; `ocorridoEm` permite ignorar eventos antigos. Filas: `pedidos-eventos` (PedidoCadastrado, PedidoAlterado, PedidoExcluido; produtor: Serviço de Pedidos) e `telemetria-veiculos` (LeituraDeTelemetriaRecebida; produtor: dispositivo OBD2/GPS). Consumidor de ambas: Serviço de Frota. Entrega "pelo menos uma vez" e sem ordem garantida — por isso o consumidor é idempotente, descarta eventos antigos e usa concorrência otimista.

8.13. Observabilidade

O gateway gera (ou aceita) um `x-correlation-id`, registra cada requisição no log com esse identificador (Application Insights) e o devolve ao navegador. O mesmo

cabeçalho é repassado ao BFF e, pelo BFF, aos microsserviços — a base para, numa próxima etapa, incluí-lo também nos logs desses serviços e seguir uma requisição pelos três Function Apps.

9. Decisões de Arquitetura

ID	Decisão	Contexto	Consequências
ADR-01	MongoDB Atlas (documento) em vez de banco relacional	Telemetria futura tem esquema variável; free tier simples; driver Node direto	Sem *joins*/constraints — integridade referencial fica na aplicação; o DER (seção 8) é conceitual, não físico
ADR-02	Static Web Apps só para o frontend; backend em 4 Function Apps no plano de Consumo	Microsserviços precisam de deploy e escala independentes; managed functions do SWA permitem só um app de API	Mais recursos para administrar; cold start em cadeia (R10)
ADR-03	Navegação por estado (`useState` em `App.jsx`) em vez de `react-router`	Apenas 2 telas; manter o exemplo simples (CV03)	Sem *deep-link*/URL por tela; migração para `react-router-dom` é trivial e isolada
ADR-04	`id` público string gerado no servidor em vez de expor o `_id` (ObjectId)	Frontend precisa de identificador legível e estável; não depender do cliente	Risco de colisão (baseado em *timestamp*, 6 dígitos) — aceito no escopo; trocar por ULID/UUID ou índice único + *retry* se necessário
ADR-05	Fallback local no microfrontend, com aviso "Modo offline" visível	O fallback silencioso mascarava falhas (R05)	O usuário sabe quando os dados não são reais
ADR-06	Azure Functions modelo v4 (código-first, sem `function.json`)	Modelo atual recomendado pela Microsoft; menos arquivos de configuração	Exige *extension bundle* `[4.*,5)` e `@azure/functions ^4`; rotas declaradas em código (`route: 'pedidos/{id}'`)

ID	Decisão	Contexto	Consequências
ADR-07	Uma camada `services/` por microfrontend, chamando só o gateway	Cada microfrontend é autônomo	Pequena duplicação entre os `services/` (aceita, R14)
ADR-08	Microfrontends com Module Federation (shell + 2 remotos), publicados no mesmo Static Web App em subpastas	Telas evoluem separadamente; URLs relativas evitam CORS e configuração por ambiente	Builds independentes; deploy hoje conjunto (um SWA); React singleton obrigatório
ADR-09	API Gateway implementado em código (Azure Function) em vez de Azure API Management	Custo zero, roda localmente, regras testáveis no CI	Rate limit por instância (R13); sem portal de desenvolvedor
ADR-10	Um BFF por tipo de cliente (BFF Web agora; BFF Mobile planejado)	Telas precisam de dados combinados de serviços diferentes	Lógica de composição fora das telas e dos serviços
ADR-11	Dois microsserviços (Pedidos e Frota) com Database per Service	Pedidos e frota mudam por motivos diferentes; a telemetria tem volume próprio	Sem joins entre bancos; dados replicados por eventos; consistência eventual (R12)
ADR-12	Colaboração por eventos em Azure Queue Storage	Barato, nativo do Functions (Queue Trigger), emulável localmente (Azurite)	Uma fila por consumidor; sem fan-out nativo — para vários consumidores, migrar para Service Bus Topics ou Event Grid
ADR-13	Clean Architecture + Vertical Slice em todos os backends	Mesma estrutura em 4 projetos facilita a manutenção e a	Mais arquivos por caso de uso

ID	Decisão	Contexto	Consequências
		verificação automática	
ADR-14	Testes de arquitetura com `node:test` e um analisador de imports próprio	Sem dependências; regras específicas do projeto (bancos, filas, gateway)	O analisador cobre `require`/`import` estáticos e dinâmicos com texto literal
ADR-15	Concorrência otimista (campo `versao`) na projeção de entregas	Mensagens da mesma entrega são processadas em paralelo	Retentativa no caso de uso; índice único em `entregas.id`
ADR-16	Publicação de eventos "melhor esforço", sem Transactional Outbox	Simplicidade no escopo acadêmico	Evento pode se perder se a fila falhar logo após a gravação (R11)

10. Requisitos de Qualidade

10.1. Árvore de Qualidade

Categoria (ISO 25010)	Subcaracterística	Prioridade	Objetivo relacionado
Confiabilidade	Tolerância a falhas / disponibilidade	Alta	Q1
Adequação de custo (contexto do projeto)	Operação em plano gratuito	Alta	Q2
Manutenibilidade	Modularidade, analisabilidade	Alta	Q3
Portabilidade	Adaptabilidade, capacidade de substituição	Média	Q4
Manutenibilidade	Testabilidade	Alta (141 testes automatizados)	Q3
Segurança	Autenticidade, confidencialidade	Média hoje / Alta no futuro	Q5
Desempenho	Comportamento temporal (*cold start*)	Baixa	—
Manutenibilidade	Modularidade verificada (testes de arquitetura)	Alta	Q3
Confiabilidade	Isolamento de falhas entre partes	Alta	Q1, Q6
Portabilidade	Implantação independente	Média	Q6

10.2. Cenários de Qualidade

ID	Atributo	Estímulo	Resposta esperada	Métrica
CQ01	Disponibilidade	Gateway ou banco indisponível	Telas continuam navegáveis com	0 tela de erro fatal; aviso

ID	Atributo	Estímulo	Resposta esperada	Métrica
		durante o uso	o aviso "Modo offline"	sempre visível
CQ02	Custo	Operar a solução durante um semestre letivo	Fatura Azure + Atlas dentro do free tier	R$ 0,00
CQ03	Automação de implantação	`push` na branch `main`	Nova versão publicada sem etapa manual	< 5 min; 100% via GitHub Actions
CQ04	Evolutibilidade	Trocar a tabela de veículos por um mapa real	Alterar só o `mfe-frota`	0 alteração no shell, no `mfe-pedidos` e no backend
CQ05	Manutenibilidade	Novo integrante precisa entender um caso de uso	Abrir uma única pasta de fatia	1 pasta por caso de uso
CQ06	Portabilidade	Trocar MongoDB ou a fila por outro serviço	Reescrever só o adaptador em `infraestrutura/`	0 alteração em domínio e casos de uso
CQ07	Isolamento de falhas	`remoteEntry.js` do `mfe-frota` indisponível	Shell e `mfe-pedidos` continuam funcionando	Falha contida na área do microfrontend
CQ08	Conformidade arquitetural	Um PR faz o domínio importar `mongodb`	CI falha e o deploy não acontece	100% dos PRs verificados
CQ09	Consistência eventual	Coordenador conclui uma entrega com 3 pedidos	Painel da frota mostra "Concluída 3/3"	< 15 s; nenhuma atualização perdida
CQ10	Segurança	Chamada direta a um microsserviço	Recusada	HTTP 401

ID	Atributo	Estímulo	Resposta esperada	Métrica
		sem chave		
CQ11	Proteção de capacidade	Mais de 120 req/min do mesmo IP	Gateway responde 429 com `Retry-After`	Nenhuma chamada excedente chega ao BFF

11. Riscos e Dívida Técnica

ID	Risco / Dívida técnica	Impacto	Mitigação / Plano
R01	Sem autenticação/autorização (`authLevel: anonymous`, CORS *)	Qualquer pessoa pode ler, criar, alterar e excluir pedidos	Parcialmente mitigado. Microsserviços e BFF exigem chave e o CORS foi restrito; ainda não há autenticação de usuário (RF14).
R02	MongoDB Atlas com *Network Access* `0.0.0.0/0`	Banco exposto à internet, protegido apenas pela credencial na `MONGODB_URI`	Restringir faixas de IP ou usar *Private Endpoint* (exige plano pago); rotacionar segredo periodicamente
R03	Segredos em variáveis `VITE_*`	Qualquer `VITE_*` vira texto público no bundle servido	Regra: `VITE_*` só para URLs públicas; segredos só no backend (*Application settings*)
R04	Sem testes automatizados	Regressões nas Functions ou no frontend passam despercebidas	Resolvido. 76 testes de unidade + 65 testes de arquitetura, executados no CI antes de todo deploy.
R05	Fallback mock mascara falhas reais	Numa demonstração, indisponibilidade de API/banco não é percebida	Mitigado. O modo offline agora é exibido na tela.
R06	Região única / sem DR; M0 sem backup configurável	Indisponibilidade regional derruba toda a solução; risco de perda de dados	Aceitável no escopo acadêmico; definir RTO/RPO e estratégia de backup se evoluir para produção

ID	Risco / Dívida técnica	Impacto	Mitigação / Plano
R07	`id` de pedido baseado em *timestamp* (6 dígitos)	Colisão sob concorrência / alto volume	Trocar por ULID/UUID, ou índice único em `id` + *retry* na inserção
R08	Grande escopo planejado ainda não implementado (OBD2, GPS, offline, mobile, alertas, histórico, relatórios, POD)	Distância entre a documentação de contexto (produto completo) e o sistema atual	*Roadmap* por *release* (seção 1.4); seções 5–7 marcam explicitamente o que é "planejado"
R09	Código legado "concluir entrega" (integração Apidog) ainda em `services/api.js`	Fora do escopo desta documentação; pode confundir a leitura do código	Resolvido. A integração com o Apidog foi removida; "concluir entrega" passou a ser atendida pelo BFF.
R10	Cold start em cadeia: uma requisição pode acordar até 3 Function Apps (gateway → BFF → serviço).	Primeira requisição após ociosidade é lenta (inclui `connect` ao Atlas)	Conexões em cache; se necessário, plano Flex Consumption com instâncias sempre prontas.
R11	Publicação de eventos sem Transactional Outbox	Pedido gravado sem evento se a fila falhar nesse instante; projeção desatualizada	Implementar outbox (coleção de eventos pendentes + publicador)
R12	Consistência eventual entre serviços	O painel da frota pode refletir uma alteração alguns segundos depois	Aceito; documentado na tela ("atualiza a cada 15 s")
R13	Rate limit em memória, por instância	Com várias instâncias, o limite efetivo é maior	Mover contagem para cache distribuído ou usar API Management
R14	Duplicação de estilos e de lógica de fallback	Mudança visual precisa ser	Biblioteca de design compartilhada,

ID	Risco / Dívida técnica	Impacto	Mitigação / Plano
	entre microfrontends	repetida	versionada
R15	Chaves de Function como segredo compartilhado	Chave vazada dá acesso direto a um serviço	Rotacionar chaves; evoluir para identidade gerenciada
R16	A projeção de entregas não tem replay	Pedidos anteriores à criação do serviço de frota só aparecem quando alterados	Script de carga inicial que publica eventos para os pedidos existentes